

Design Document

Basic Cyber Physical System Attack and Defense Testbed

Project team	Version and date
Karla Daley and Zachary Lavoie Faculty Advisor and Client Sneha Sudhakaran	Version 1.0 September 11, 2026

1 Introduction

1.1 Purpose

This document describes how the software and connected hardware will be organized to satisfy the approved requirements for the testbed. It is written for the two-person development team, the faculty advisor, future maintainers, and reviewers who need to understand how PLC data moves through the system and how controlled attacks will be observed.

1.2 Scope

The design covers three vendor PLCs connected through an isolated Ethernet network, physical sensors, an HMI (attack workstation), a historian with a browser-based user interface, a PLC profile database, Python test and attack scripts, network capture tools, and an ESP32 sensor-interception module. The historian interface is created by the team. The PLC programming interfaces supplied by manufacturers are used only to configure and diagnose the controllers.

1.3 Document Organization

The document presents the system context, architecture, data design, component behavior, historian interface, and traceability to the requirements. The architecture, data-flow, and interface diagrams are identified for creation in draw.io.

1.4 References

Project Plan for the Basic Cyber Physical System Attack and Defense Testbed; System Requirements Specification Revision 0.2; Basic Cyber Physical System Attack and Defense Testbed Level Test Plan Version 1.0; IEEE Std 1016-2009; and the provided SWDD template.

1.5 Definitions

Term	Meaning
PLC	Programmable Logic Controller that reads sensors and controls a physical process.
Historian	Team-built service and interface that displays and stores PLC and sensor data.
HMI (attack workstation)	Computer that runs controlled Python attack scripts on the isolated test network.
PLC profile	Stored PLC identity, protocol, address, tag, input/output, and sensor information.
Test run number	Unique number used to connect one test with its records, capture, and result.

2 System Overview

The testbed connects Allen-Bradley, Siemens, and or Modicon PLCs to one managed laboratory network. Each PLC reads at least one sensor. A protocol-specific adapter receives the PLC data and converts it into a common format for the historian. The historian stores the data and presents it via a browser, allowing an industry user to confirm device status, live values, test activity, and recorded events.

Normal operation. Sensors update the PLCs, the adapters read the PLC values, and the historian stores and displays the results.

Cyber test. The HMI (attack workstation) executes a selected Python script on the PLC, while the historian and network monitor record the response.

Hardware test. The ESP32 interception module interrupts or changes a sensor signal before it reaches a PLC, allowing the physical and recorded effects to be compared.

3 System Architecture

3.1 Architectural Design

The system uses a layered architecture, separating vendor-specific PLC communication from data storage, monitoring, and testing. This allows a new PLC profile or protocol adapter to be added without rebuilding the historian interface. The HMI (attack workstation) remains separate from the historian, so testing and monitoring have different responsibilities.

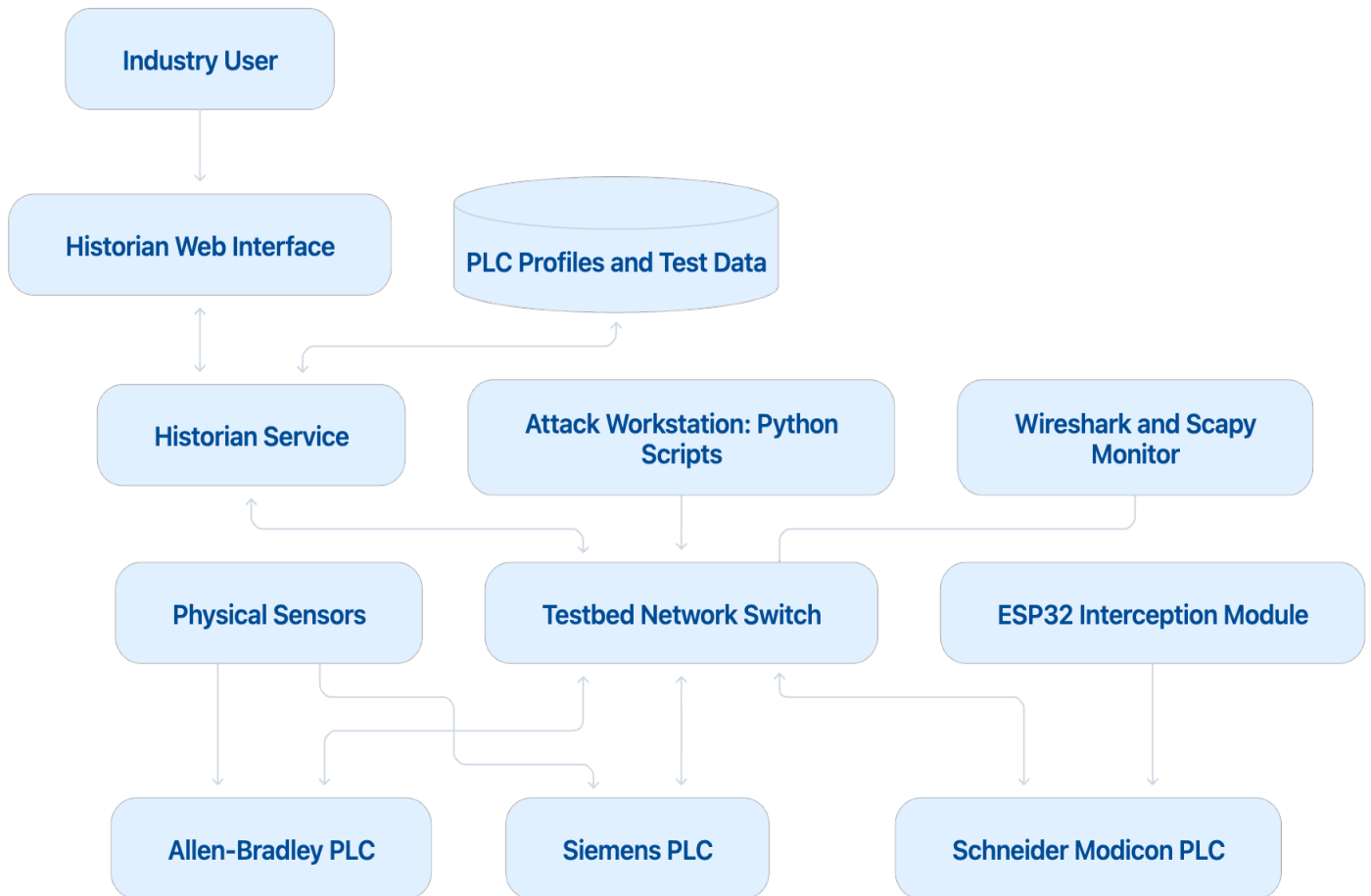


Figure 1 System Architecture

Design component	Responsibility	Main connection
Historian web UI	Shows live PLC and sensor data, test status, and saved events.	Historian service
Historian service	Reads normalized PLC data, stores records, and supplies the UI.	Adapters and database
PLC adapter layer	Handles EtherNet/IP, PROFINET, and Modbus TCP differences.	PLCs and historian
PLC profile database	Stores device identity, protocol, network, tag, and sensor mappings.	Historian and adapters
HMI (attack workstation)	Runs approved Python scripts that simulate internal or external attacks.	Isolated Ethernet network
Network monitor	Captures traffic using Wireshark or Scapy.	Ethernet switch
ESP32 module	Passes, interrupts, or changes a selected sensor signal.	Sensor and PLC input

3.2 Decomposition and Data Flow

The design is divided into acquisition, storage, presentation, testing, and physical-interception functions. The PLC adapters acquire values. The historian service validates the PLC identity using its stored profile, attaches a timestamp and test run number, and writes the record to the database. The browser interface requests current and historical records from the historian service rather than communicating directly with the PLCs.

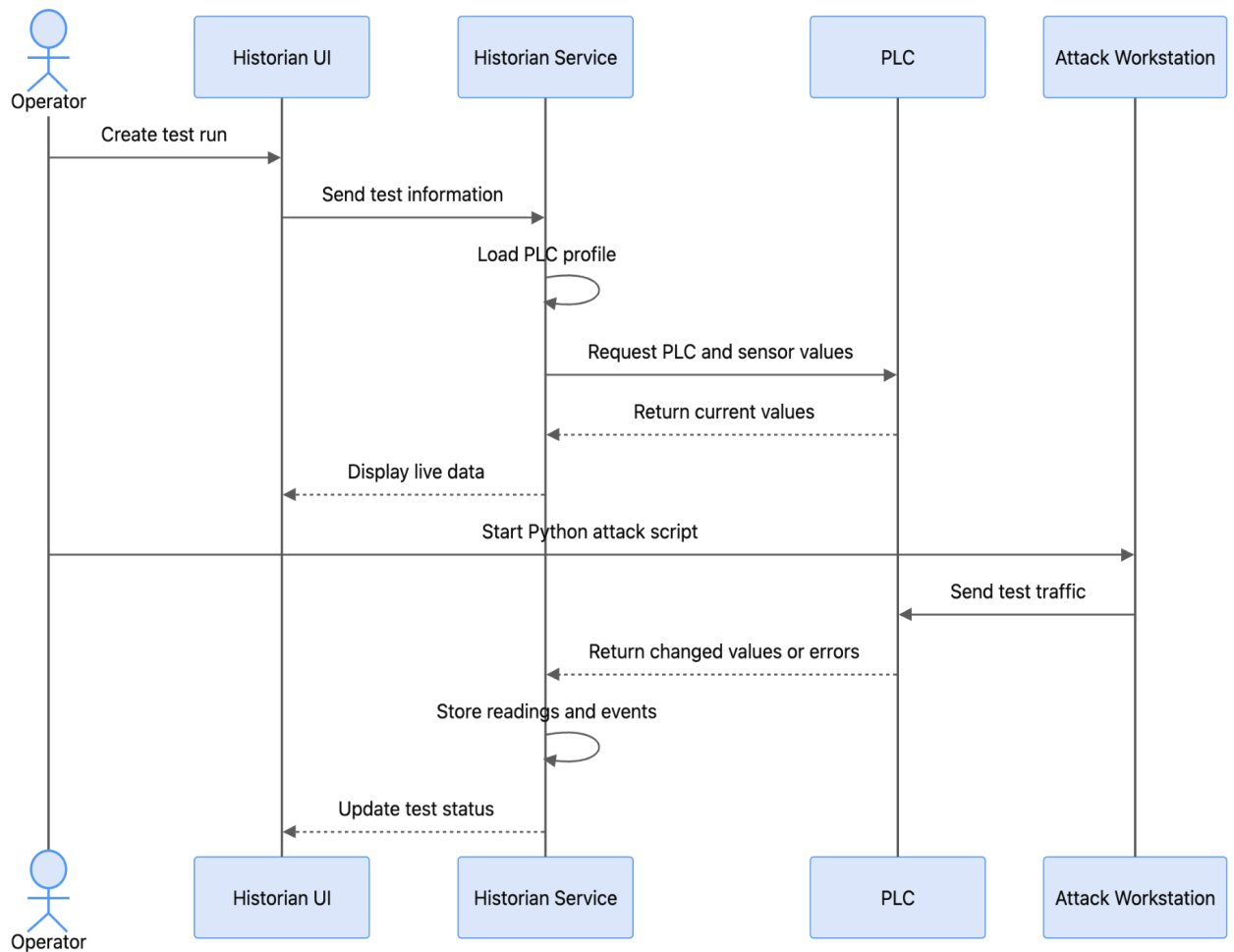


Figure 2 Test Data Flow

3.3 Deployment Design

Location	Installed items	Design note
Historian mini computer	Historian service, browser UI, database, and shared project services	Non-vendor-specific services may run in Docker.
HMI (attack mini computer)	Python attack scripts, Wireshark, Scapy, and approved security tools	Connected only to the isolated test network.
PLC engineering computer	Studio 5000 or RSLogix, TIA Portal or STEP 7, EcoStruxure or SoMachine Basic	Vendor software configures the PLCs; it is not developed by the team.
Physical testbed	Three PLCs, switches, sensors, DIN rail, and ESP32 relay module	Wiring follows the documented input/output mapping.

3.4 Design Rationale

Common adapter layer. Each PLC uses a different protocol, so adapters convert vendor-specific values into one historian format. This supports comparison and future plug-and-play expansion.

Separate monitoring and attack systems. The historian observes and records activity, while the HMI (attack workstation) produces the controlled test stimulus. Therefore, a failure or restart of the HMI (attack workstation) does not control historian storage.

Database-driven configuration. PLC details are stored as profiles rather than being written repeatedly in Python scripts. This reduces setup effort and makes changes easier to maintain.

Isolated deployment. All attacks remain on the laboratory network, protecting university and public networks while still allowing realistic industrial traffic to be studied.

4 Data Design

Data from tests will be stored in timestamped folders containing all relevant logs for each test. This is done to reduce overhead & allow easy analysis of a specific test, in which all original logs from all systems can be found and compared. This keeps things intuitive to users, as things like network traffic will be in a standard PCAP.

Data item	Important fields	Purpose
PLC Profile	PLC ID, manufacturer, model, protocol	Identifies a PLC and supplies plug-and-play settings.
Sensor Profile	Sensor type, Connection configuration	Stores the measured physical input.

5 Component Design

Component	Processing design	Failure response
PLC Adapter Manager	Loads the selected PLC profile, opens the correct protocol connection, and converts received values to the common historian format.	Marks the PLC disconnected and reports the connection problem.
Historian Collector	Reads adapter updates, adds time and test run information, and writes readings and events to the database.	Keeps existing records and identifies missing or stale data.
Historian API	Answers UI requests for PLC profiles, live readings, test runs, and event history.	Returns a clear error without changing PLC operation.
Historian Web UI	Displays current state, PLC details, active tests, and historical records in the browser.	Shows unavailable data instead of presenting old values as current.
Python Test Runner	Loads an approved script and PLC profile, assigns a test run number, executes the steps, and records the result.	Stops the script, records Failed or Stopped, and keeps collected evidence.

Component	Processing design	Failure response
Network Capture Service	Starts before a test, records traffic, and saves the capture under the test run number.	Reports a missing capture while allowing other records to remain.
ESP32 Interception Controller	Receives an approved pass, interrupt, or alter command for the selected sensor channel.	Returns the channel to normal pass-through when the test ends.

5.1 Main Test Sequence

Prepare. The user signs in, selects a PLC profile and Python script, and confirms that the historian, PLC, sensor, and capture tool are connected.

Record baseline. The historian saves the normal PLC state and sensor value, then the system creates the next test run number.

Execute and observe. The script or ESP32 module applies the approved test action while the historian and network monitor record the response.

Finish. The test is marked Completed, Stopped, or Failed; normal operation is restored; and all records remain linked by the test run number.

6 Human Interface Design

6.1 Historian Interface

The historian interface will be a browser-based dashboard built with HTML, CSS, and JavaScript and supported by a Python service. Its purpose is to help an industry user quickly identify which PLC is connected, confirm that sensors are working, observe changes during a test, and review saved records. The interface does not replace the vendor PLC programming software.

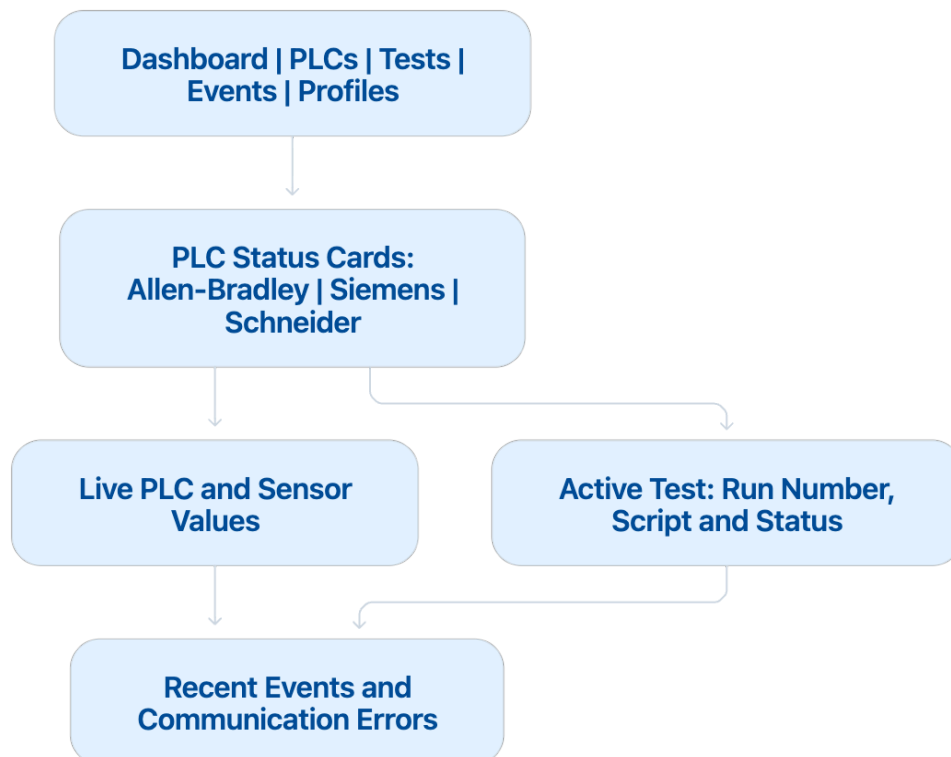


Figure 3 Historian Dashboard Wireframe

6.2 Screen Objects and Actions

Screen or object	Information shown	User action
Login	Username and password fields plus login result	Enter authorized credentials.
Overview Dashboard	Three PLC connection cards, protocol, last update, sensor state, and active test	Select a PLC or active test for details.
PLC Details	PLC profile, IP address, mapped tags, connected sensors, and recent values	Review configuration and live data.
Test Monitor	Test run number, selected script, target PLC, baseline value, current value, and status	Start or stop an approved test and view its progress.
Event History	Timestamped PLC changes, sensor changes, attack events, communication problems, and results	Filter by PLC, test run number, date, or event type.
PLC Profiles	Stored manufacturer, model, protocol, network, tag, and sensor information	Add or update an authorized PLC profile.

7 Requirements Traceability Matrix

Design area	Requirements addressed	Design evidence
Network and PLC adapters	IR-001, IR-002, FR-001, FR-002, PER-001	Three protocol adapters, Ethernet topology, and PLC profile loading
Historian and database	IR-003, IR-005, IR-006, FR-003, FR-005, FR-009, FR-012, PER-002-PER-004	Collector, relational data model, API, live dashboard, and event history
Attack and test services	IR-004, IR-007, FR-004, FR-006-FR-011, REL-001	Python test runner, test run numbering, HMI (attack workstation), and ESP32 controller
Security and recovery	REL-002, AVL-001, SEC-001-SEC-005	Isolated network, login, protected credentials, approved targets, and retained records
Maintenance and deployment	MNT-001-MNT-003, PRT-001-PRT-003	PLC documentation, database profiles, Docker services, browser UI, and editable settings

8 Implementation and Verification Plan

The modular design allows the two team members to work in parallel. Karla will lead PLC configuration, sensor and input/output mapping, and the hardware-interception path. Zachary will lead network and protocol integration, historian communication, packet capture, and cyber test scripts. Both members will review the historian interface, database integration, requirements traceability, and final demonstrations.

Verification follows the Level Test Plan: components are tested individually, connected components are tested together, the full system is tested on an isolated network, and milestone demonstrations provide evidence of acceptance. Each result is compared with the expected PLC state, the historian record, the sensor response, and the network capture.